Reaktive Systeme
(Formelsammlung v2627.1)

Uwe Nestmann
Technische Universität Berlin

2. Oktober 2026

Zusammenfassung

Das empfehlenswerte Lehrbuch *Reactive Systems* [AILS07] bildet die Grundlage eines Moduls, das seit 2013 im Bachelor Informatik an der Technischen Universität Berlin angeboten wird. Bis 2014 war es Pflichtmodul mit der Bezeichnung *Spezifikation und Semantik* (bzw. *Theoretische Grundlagen der Informatik 4*). Ab 2015 wird es als Wahlpflichtmodul unter dem Namen *Reaktive Systeme* geführt.

Das vorliegende Dokument listet die für das Modul notwendigen Definitionen und Theoreme des genannten Lehrbuchs in deutscher Sprache. Das Dokument ist bei weitem keine Übersetzung des kompletten Lehrbuchs ins Deutsche, da es auf Erklärungen, Texte und Beispiele verzichtet. Es dient eher als schnelles Nachschlagewerk und hält sich weitgehend an die Notationen des Lehrbuchs.

Soweit möglich, verwenden wir bei Übereinstimmungen mit dem Lehrbuch die exakt gleichen Referenz-Zähler. Oftmals werden Definitionen und Aussagen im Lehrbuch jedoch im Fließtext eingeführt; in diesen Fällen verwenden wir einen separaten Zähler.

Literatur

[AILS07] Luca Aceto, Anna Ingólfsdóttír, Kim Guldstrand Larsen, and Jiří Srba. *Reactive Systems — Modelling, Specification and Verification*. Cambridge University Press, 2007.

Inhaltsverzeichnis

2 Die Sprache CCS **3**

3 Verhaltensäquivalenzen **9**

4 Fixpunkttheorie **16**

5 Hennessy-Milner-Logik **21**

6 HML mit Rekursion **24**

2 Die Sprache CCS

2.1 Definition (LTS; Definition 2.1 in [AILS07])
Ein *beschriftetes Transitionssystem* (LTS) ist ein Tripel[1]

$$(\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$$

mit:

- Proc ist eine Menge von *Zuständen* (oder *Prozessen*);
- Act ist eine Menge von *Aktionen* (oder *Beschriftungen*); und
- $\mathsf{Tran} \subseteq \mathsf{Proc} \times \mathsf{Act} \times \mathsf{Proc}$ ist eine *Transitionsrelation*.[2]

Wir schreiben auch $s \xrightarrow{\alpha}_{\mathsf{Tran}} s'$ anstelle von $(s, \alpha, s') \in \mathsf{Tran}$.
Wir schreiben auch $s \xrightarrow{\alpha} s'$ anstelle von $s \xrightarrow{\alpha}_{\mathsf{Tran}} s'$.
Wir schreiben $s \not\xrightarrow{\alpha}$, falls es kein s' gibt mit $s \xrightarrow{\alpha} s'$.

Ein LTS ist *endlich*, falls sowohl Proc als auch Act endlich sind;
ansonsten nennen wir es *unendlich*.

Interpretation als Graph:

Ein *Transitionsgraph* ist ein Sextupel[3] $(\mathsf{Proc}, \mathsf{E}, s, t, \mathsf{Lab}, \ell)$ mit:

- Proc ist die Menge der *Knoten*;
- E ist eine Menge von *Kanten*;
- $s : \mathsf{E} \to \mathsf{Proc}$ und $t : \mathsf{E} \to \mathsf{Proc}$ sind Abbildungen
  auf die jeweilige *Quelle* $s(e)$
  und das jeweilige *Ziel* $t(e)$ einer Kante $e \in \mathsf{E}$;
- Lab ist eine Menge von *Kantenbeschriftungen*;
- $\ell : \mathsf{E} \to \mathsf{Lab}$ ist eine *surjektive* Abbildung;
- $$\forall e_1, e_2 \in \mathsf{E} \;.\; e_1 \neq e_2 \to \Big(s(e_1) \neq s(e_2) \vee t(e_1) \neq t(e_2) \vee \ell(e_1) \neq \ell(e_2)\Big)$$

Wenn im Kontext nicht anders angegeben, setzen wir $\mathsf{Act} \triangleq \mathsf{Lab}$.[4]
Die Transitionsrelation ergibt sich als $\mathsf{Tran} \triangleq \{\, (s(e), \ell(e), t(e)) \mid e \in \mathsf{E} \,\}$.

2.2 Definition (Alternative Transitionsrelation)
Die Relation $\mathsf{Tran} \subseteq \mathsf{Proc} \times \mathsf{Act} \times \mathsf{Proc}$ wird auch durch die Familie

$$\left(\xrightarrow{\alpha}\right)_{\alpha \in \mathsf{Act}}$$

mit $\xrightarrow{\alpha} \triangleq \{\, (s, t) \mid (s, \alpha, t) \in \mathsf{Tran} \,\}$ dargestellt.

---

[1] Wir bevorzugen hier die Variante aus Remark 2.2 [AILS07]

[2] Die nur semiformale Schreibweise $\mathsf{Proc} \times \mathsf{Act} \times \mathsf{Proc}$ ist eine in vielen Lehrbüchern „geduldete“ Abkürzung für $\bigtimes_{i \in \{1,2,3\}} T_i$ mit $T_1 = \mathsf{Proc}, T_2 = \mathsf{Act}, T_3 = \mathsf{Proc}$.

[3] Hier eine spezielle Form von beschrifteten gerichteten Multigraphen.

[4] Für CCS (Definition 2.8) werden wir davon abweichen.

2.3 Definition (Abkürzungen; Notation 2.1 in [AILS07])
Wir schreiben:

- $s \xrightarrow{\varepsilon} s$, für alle $s \in \mathsf{Proc}$;
- $s \xrightarrow{\alpha w} s'$ für alle $s, s' \in \mathsf{Proc}$, $\alpha \in \mathsf{Act}$ und $w \in \mathsf{Act}^*$, falls es $t \in \mathsf{Proc}$ gibt mit $s \xrightarrow{\alpha} t$ und $t \xrightarrow{w} s'$;
- $s \to^* s'$ für alle $s, s' \in \mathsf{Proc}$, falls es $w \in \mathsf{Act}^*$ gibt mit $s \xrightarrow{w} s'$;
- $s \to s'$ für alle $s, s' \in \mathsf{Proc}$, falls es $\alpha \in \mathsf{Act}$ gibt mit $s \xrightarrow{\alpha} s'$;
- $s \xrightarrow{\alpha}$ für alle $s \in \mathsf{Proc}$, falls es $s' \in \mathsf{Proc}$ gibt mit $s \xrightarrow{\alpha} s'$;
- $s \to$ für alle $s \in \mathsf{Proc}$, falls es $s' \in \mathsf{Proc}$ und $\alpha \in \mathsf{Act}$ gibt mit $s \xrightarrow{\alpha} s'$.

2.4 Definition (Erreichbare Zustände; Definition 2.2 in [AILS07])
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Seien $s, s' \in \mathsf{Proc}$.
Dann heißt s' *erreichbar* ab s, falls $s \to^* s'$.

- $\mathrm{Reach}(s) \triangleq \{\, s' \in \mathsf{Proc} \mid s \to^* s' \,\}$

ist der *von* s *aus erreichbare Zustandsraum*. Mit

- $\mathsf{Tran}_{\mathrm{Reach}(s)} \triangleq \{\, (s', \alpha, s'') \in \mathsf{Tran} \mid s' \in \mathrm{Reach}(s) \,\}$
- $\mathsf{Act}_{\mathrm{Reach}(s)} \triangleq \left\{\, \alpha \in \mathsf{Act} \mid (\cdot, \alpha, \cdot) \in \mathsf{Tran}_{\mathrm{Reach}(s)} \,\right\}$

ergibt sich das *von* s *aus erreichbare LTS*
als $\mathrm{LTS}(s) \triangleq \big(\mathrm{Reach}(s), \mathsf{Act}_{\mathrm{Reach}(s)}, \mathsf{Tran}_{\mathrm{Reach}(s)}\big)$.

2.5 Definition (Syntax)

- $\mathcal{A}$ ist eine abzählbar unendlich große Menge von *(Kanal-)Namen*;
- $\overline{\mathcal{A}} \triangleq \{\, \bar{a} \mid a \in \mathcal{A} \,\}$ ist die Menge der *Co-Namen*;
- $\mathcal{L} \triangleq \mathcal{A} \cup \overline{\mathcal{A}}$ ist die Menge der *Beschriftungen*; wir benutzen die Metavariablen $a, b, \ldots$;
- $\mathsf{Act} \triangleq \mathcal{L} \cup \{\, \tau \,\}$ ist die Menge der *Aktionen*. wir benutzen die Metavariablen $\alpha, \beta, \ldots$.

Es gilt $\bar{\bar{a}} = a$.
Wir schreiben $\overline{L} \triangleq \{\, \bar{a} \mid a \in L \,\}$ für beliebige $L \subseteq \mathcal{L}$.
Es gibt kein $\bar{\tau}$.

- $\mathcal{K}$ ist eine abzählbar unendlich große Menge von *Prozesskonstanten*; wir benutzen die Metavariablen $A, \ldots, K, \ldots$ sowie die konkreten Konstanten $\mathbf{A}, \ldots, \mathbf{K}, \ldots$

2.6 Definition (Syntax; Definition 2.3 in [AILS07])
Die Menge $\mathcal{P}$ der CCS-Ausdrücke ist gegegeben durch folgende Grammatik:

$$P \ ::= \ \mathbf{K} \ \big| \ \alpha.P \ \big| \ \textstyle\sum_{i\in I} P_i \ \big| \ P \mid P \ \big| \ P[f] \ \big| \ P \setminus A$$

- $\mathbf{K} \in \mathcal{K}$ ist eine Prozesskonstante;
- $\alpha \in \mathsf{Act}$ ist eine Aktion;
- I ist eine (potenziell abzählbar unendlich große) Indexmenge;
- $f : \mathcal{A} \to \mathcal{A}$ ist eine *Umbenennungsfunktion* auf Namen;
- $A \subseteq \mathcal{A}$ ist eine Menge von Namen;

Für jede benutzte Konstante $\mathbf{K}$ muss eine Definition der Form

$$\mathbf{K} \stackrel{\text{def}}{=} P$$

angegeben werden. Als Abkürzungen gelten:

$$\mathbf{0} \triangleq \sum_{i\in\emptyset} P_i \qquad\qquad P_1 + P_2 \triangleq \sum_{i\in\{1,\,2\}} P_i$$

Es gilt Operatorenvorrang in absteigender Folge entsprechend:

- Restriktion $P \setminus A$ und Umbenennung $P[f]$,
- Präfix $\alpha.P$,
- Parallelkomposition $P_1 \mid P_2$, und
- Summe $\sum_{i\in I} P_i$.

2.7 Definition (Umbenennung) Oft benutzen wir die Schreibweise

$$P[b_1/a_1, \ldots, b_n/a_n]$$

wobei die $a_1, \ldots, a_n$ paarweise verschieden sein müssen
mit $1 \leqslant n \in \mathbb{N}$ und $\{a_1, \ldots, a_n, b_1, \ldots, b_n\} \subseteq \mathcal{A}$.

Dies entspricht dem CCS-Ausdruck $P[f]$ für $f : \mathcal{A} \to \mathcal{A}$ mit:

$$f(a) \triangleq \begin{cases} b_i & \text{falls } a = a_i \text{ für } i \in [1, n] \\ a & \text{falls } a \in \mathcal{A} \setminus \{a_1, \ldots, a_n\} \end{cases}$$

Ausgehend von $f : \mathcal{A} \to \mathcal{A}$ definieren wir $f^* : \mathsf{Act} \to \mathsf{Act}$ durch:

$$\begin{array}{lcll} f^*(a) & \triangleq & f(a) & \text{für alle } a \in \mathcal{A} \\ f^*(\overline{a}) & \triangleq & \overline{f(a)} & \text{für alle } a \in \mathcal{A} \\ f^*(\tau) & \triangleq & \tau & \end{array}$$

2.8 Definition (Semantik)
$(\mathcal{P}, \mathsf{Act}, \mathsf{Tran})$ wird wie folgt bestimmt:

- $\mathcal{P}$ ist die Menge der CCS-Prozesse gemäß Definition 2.6.
- Act ist die Menge von Aktionen gemäß Zusatz 2.5.
- Tran ist definiert durch die folgenden acht Ableitungs-Regeln:

$$\text{ACT} \quad \frac{}{\alpha.P \xrightarrow{\alpha} P}$$

$$\text{SUM}_k \quad \frac{P_k \xrightarrow{\alpha} P'_k}{\sum_{i\in I} P_i \xrightarrow{\alpha} P'_k} \quad \text{für } k \in I$$

$$\text{CON} \quad \frac{P \xrightarrow{\alpha} P'}{\mathbf{K} \xrightarrow{\alpha} P'} \quad \text{für } \mathbf{K} \stackrel{\text{def}}{=} P$$

$$\text{COM1} \quad \frac{P \xrightarrow{\alpha} P'}{P \mid Q \xrightarrow{\alpha} P' \mid Q}$$

$$\text{COM2} \quad \frac{Q \xrightarrow{\alpha} Q'}{P \mid Q \xrightarrow{\alpha} P \mid Q'}$$

$$\text{COM3} \quad \frac{P \xrightarrow{a} P' \qquad Q \xrightarrow{\bar{a}} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'}$$

$$\text{RES} \quad \frac{P \xrightarrow{\alpha} P'}{P \setminus A \xrightarrow{\alpha} P' \setminus A} \quad \begin{array}{ll} \text{für} & \alpha = \tau \\ \text{oder} & \alpha \neq \tau \wedge \{\alpha, \overline{\alpha}\} \cap A = \emptyset \end{array}$$

$$\text{REL} \quad \frac{P \xrightarrow{\alpha} P'}{P[f] \xrightarrow{f^*(\alpha)} P'[f]}$$

Die Semantik eines CCS-Prozesses $P \in \mathcal{P}$ im Kontext des LTS $(\mathcal{P}, \mathsf{Act}, \mathsf{Tran})$ basiert auf dem erreichbaren $\text{LTS}(P) = (\text{Reach}(P), \mathsf{Act}_{\text{Reach}(P)}, \mathsf{Tran}_{\text{Reach}(P)})$ aus Definition 2.4: Wir definieren sie davon abweichend als das LTS

$$(\text{Reach}(P), \mathsf{Act}_P, \mathsf{Tran}_{\text{Reach}(P)})$$

mit $\mathsf{Act}_P \triangleq (\text{Lab} \cup \overline{\text{Lab}}) \cup \{\tau\}$ für $\text{Lab} \triangleq \mathsf{Act}_{\text{Reach}(P)} \setminus \{\tau\}$, so dass – neben τ – zu allen Aktionen ($\neq\tau$) auch ihre Co-Aktion in Act vorhanden sein muss.

2.9 Definition (VP-CCS Syntax)
Die Menge $\mathcal{P}^{VP}$ der VP-CCS-Ausdrücke ist gegegeben durch:

$$P ::= \sum_{i\in I} P_i \mid P \mid P \mid P[f] \mid P \setminus A \\ \mid \mathbf{K}(e_1, \ldots, e_n) \mid \overline{a}(e).P \mid a(x).P \mid \tau.P \mid \textbf{if } \phi \textbf{ then } P \textbf{ else } P$$

- $e, e_1 \ldots, e_n \in \mathcal{E}$ sind *Ausdrücke* (auch über Variablen) zur Berechnung von *Datenelementen*;
- $v \in \mathcal{V} \subseteq \mathcal{E}$ ist die Menge von berechneten Daten bzw. *Werten*;
- $x \in \mathcal{X}$ ist eine *Variable*;
- $\phi \in \mathcal{B} \subseteq \mathcal{E}$ sind *bool'sche Ausdrücke*, mit $\{ \text{ true}, \text{ false } \} \subseteq \mathcal{V}$;
- $\text{eval} : \mathcal{E} \to \mathcal{V}$ ist eine terminierend berechenbare *Auswertefunktion*;
- alle weiteren Komponenten wie in Definition 2.6;
- die Mengen $\mathcal{X}$, $\mathcal{V}$, $\mathcal{A}$, $\overline{\mathcal{A}}$, $\{\tau\}$ sind paarweise disjunkt.

Für jede benutzte Konstante **K** muss eine Definition der Form

$$\mathbf{K}(x_1, \ldots, x_n) \stackrel{\text{def}}{=} P$$

angegeben werden, die in der Parameterliste alle „frei vorkommenden" (und paarweise verschiedenen!) Variablen $x_1, \ldots, x_n$ in P beinhaltet.

2.10 Definition (Substitution) Sei $X \triangleq \{ x_1, \ldots, x_n \} \subseteq \mathcal{X}$ mit $n \geqslant 1$ eine Menge paarweise verschiedener Variablen. Die Schreibweise

$$P\{ v_1/x_1, \ldots, v_n/x_n \}$$

mit $\{ v_1, \ldots, v_n \} \subseteq \mathcal{V}$ entspricht einer Ersetzungsfunktion auf dem CCS-Ausdruck P (als Meta-Operation außerhalb des Kalküls[5]), die alle „freien" Vorkommen von x_i in P konsistent durch das jeweilige v_i ersetzt.

2.11 Definition (VP-CCS Semantik)
$\left(\mathcal{P}^{VP}, \mathsf{Act}^{VP}, \mathsf{Tran}^{VP}\right)$ wird bestimmt wie folgt:

- $\mathcal{P}^{VP}$ ist die Menge der VP-CCS-Prozesse gemäß Definition 2.9.
- $\mathsf{Act}^{VP} \triangleq \{\tau\} \cup \{ \overline{a}(v), a(v) \mid a \in \mathcal{A}, v \in \mathcal{V} \}$ ist die Menge von VP-Aktionen mit $\overline{a(v)} = \overline{a}(v)$ und $\overline{\overline{a}(v)} = a(v)$.
- $f^{VP} : \mathsf{Act}^{VP} \to \mathsf{Act}^{VP}$ ist definiert durch:

$$\begin{array}{lcll} f^{VP}(\tau) & \triangleq & \tau & \\ f^{VP}(\overline{a}(v)) & \triangleq & \overline{f(a)}(v) & \text{für alle } a \in \mathcal{A} \text{ und } v \in \mathcal{V} \\ f^{VP}(a(v)) & \triangleq & f(a)(v) & \text{für alle } a \in \mathcal{A} \text{ und } v \in \mathcal{V} \end{array}$$

- Tran^{VP} ist definiert durch die folgenden Ableitungs-Regeln:

[5] Im Gegensatz zur Umbenennungsfunktion, die Teil der Syntax des Kalküls ist.

$$\text{OUT} \quad \frac{}{\overline{a}(e).P \xrightarrow{\overline{a}(v)} P} \quad \text{für } \mathrm{eval}(e) = v$$

$$\text{INP} \quad \frac{}{a(x).P \xrightarrow{a(v)} P\{v/x\}} \quad \text{für } v \in \mathcal{V}$$

$$\text{TAU} \quad \frac{}{\tau.P \xrightarrow{\tau} P} \qquad \text{COM} \quad \frac{P \xrightarrow{\alpha} P' \qquad Q \xrightarrow{\overline{\alpha}} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'} \quad \text{für } \alpha \neq \tau$$

$$\text{CON} \quad \frac{P\{v_1/x_1, \cdots, v_n/x_n\} \xrightarrow{\alpha} P'}{\mathbf{K}(e_1, \ldots, e_n) \xrightarrow{\alpha} P'} \quad \begin{array}{l} \text{für } \mathbf{K}(x_1, \ldots, x_n) \stackrel{\text{def}}{=} P \\ \text{und } v_i = \mathrm{eval}(e_i) \\ \text{für alle } 1 \leqslant i \leqslant n \end{array}$$

$$\text{TRUE} \quad \frac{P \xrightarrow{\alpha} P'}{\textbf{if } \phi \textbf{ then } P \textbf{ else } Q \xrightarrow{\alpha} P'} \quad \text{für } \mathrm{eval}(\phi) = \mathrm{true}$$

$$\text{FALSE} \quad \frac{Q \xrightarrow{\alpha} Q'}{\textbf{if } \phi \textbf{ then } P \textbf{ else } Q \xrightarrow{\alpha} Q'} \quad \text{für } \mathrm{eval}(\phi) = \mathrm{false}$$

$$\text{SUM}_k \quad \frac{P_k \xrightarrow{\alpha} P'_k}{\sum_{i \in I} P_i \xrightarrow{\alpha} P'_k} \quad \text{für } k \in I$$

$$\text{COM1} \quad \frac{P \xrightarrow{\alpha} P'}{P \mid Q \xrightarrow{\alpha} P' \mid Q}$$

$$\text{COM2} \quad \frac{Q \xrightarrow{\alpha} Q'}{P \mid Q \xrightarrow{\alpha} P \mid Q'}$$

$$\text{RES} \quad \frac{P \xrightarrow{\alpha} P'}{P \setminus A \xrightarrow{\alpha} P' \setminus A} \quad \text{falls } \alpha \in \{\tau\} \cup \{\, \overline{a}(v),\ a(v) \mid a \in \mathcal{A} \setminus A,\ v \in \mathcal{V} \,\}$$

$$\text{REL} \quad \frac{P \xrightarrow{\alpha} P'}{P[f] \xrightarrow{f^{VP}(\alpha)} P'[f]}$$

3 Verhaltensäquivalenzen

3.1 Definition (Äquivalenzrelation; Definition 3.1 in [AILS07])
Sei X eine Menge.
Sei $\mathcal{R} \subseteq X \times X$ eine binäre homogene Relation über X.
Wir schreiben auch $x \mathcal{R} y$, falls $(x, y) \in \mathcal{R}$.
$\mathcal{R}$ heißt *Äquivalenz[relation]*,
falls folgende Eigenschaften erfüllt sind:

- $\mathcal{R}$ ist *reflexiv*, d.h.
  für alle $x \in X$ gilt:
  $(x, x) \in \mathcal{R}$;
- $\mathcal{R}$ ist *symmetrisch*, d.h.
  für alle $x, y \in X$ gilt:
  wenn $(x, y) \in \mathcal{R}$, dann auch $(y, x) \in \mathcal{R}$;
- $\mathcal{R}$ ist *transitiv*, d.h.
  für alle $x, y, z \in X$ gilt:
  wenn $(x, y) \in \mathcal{R}$ und $(y, z) \in \mathcal{R}$, dann auch $(x, z) \in \mathcal{R}$.

Spuren

3.2 Definition (Traces)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Seien

- $k \in \mathbb{N}$
- $s_0, \ldots, s_k \in \mathsf{Proc}$
- $\alpha_1, \ldots, \alpha_k \in \mathsf{Act}$

so dass für alle $1 \leqslant j \leqslant k$ eine Transition $s_{j-1} \xrightarrow{\alpha_j} s_j$ existiert.
Dann heißt $\alpha_1 \cdot \ldots \cdot \alpha_k \in \mathsf{Act}^*$ eine *Spur* bzw. ein *Trace* in von s_0 in T.

$$\mathrm{Traces}(s) \triangleq \{\, w \in \mathsf{Act}^* \mid w \text{ ist Trace von } s \text{ in } T \,\}$$

3.3 Definition (Trace-Äquivalenz)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Seien $s_1, s_2 \in \mathsf{Proc}$.
Dann heißen s_1 und s_2 *Trace-äquivalent*, wenn:

$$\mathrm{Traces}(s_1) = \mathrm{Traces}(s_2)$$

Starke Bi-/Simulation

3.4 Definition (Starke Simulation) [6]
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{R} \subseteq \mathsf{Proc} \times \mathsf{Proc}$.
$\mathcal{R}$ heißt *(starke) Simulation*, wenn gilt:
für alle $(s_1, s_2) \in \mathcal{R}$,
für alle $\alpha \in \mathsf{Act}$,
für alle $s_1' \in \mathsf{Proc}$
wenn $s_1 \xrightarrow{\alpha} s_1'$,
dann gibt es s_2' mit $s_2 \xrightarrow{\alpha} s_2'$ mit $(s_1', s_2') \in \mathcal{R}$.
Seien $s_1, s_2 \in \mathsf{Proc}$.

- s_1 *wird von* s_2 *(stark) simuliert*, geschrieben $s_1 \lesssim s_2$,
  falls es eine (starke) Simulation $\mathcal{R}$ gibt mit $(s_1, s_2) \in \mathcal{R}$.

- s_1 *und* s_2 *simulieren sich (stark) gegenseitig*, geschrieben $s_1 \simeq s_2$,
  falls es zwei (starke) Simulationen $\mathcal{R}_1$ und $\mathcal{R}_2$ gibt mit

$$(s_1, s_2) \in \mathcal{R}_1 \text{ und } (s_2, s_1) \in \mathcal{R}_2 \ .$$

3.5 Definition (Alternative Definition für Bisimulation)
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{R} \subseteq \mathsf{Proc} \times \mathsf{Proc}$.
$\mathcal{R}$ heißt *(starke) Bisimulation*,
wenn sowohl $\mathcal{R}$ als auch $\mathcal{R}^{-1}$ eine (starke) Simulation ist.

3.6 Definition (Bisimulation; Definition 3.2 in [AILS07])
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{R} \subseteq \mathsf{Proc} \times \mathsf{Proc}$.
$\mathcal{R}$ heißt *(starke) Bisimulation*, wenn gilt:

1. für alle $(s_1, s_2) \in \mathcal{R}$,
   für alle $\alpha \in \mathsf{Act}$,
   für alle $s_1' \in \mathsf{Proc}$
   wenn $s_1 \xrightarrow{\alpha} s_1'$,
   dann gibt es s_2' mit $s_2 \xrightarrow{\alpha} s_2'$ mit $(s_1', s_2') \in \mathcal{R}$.

2. für alle $(s_1, s_2) \in \mathcal{R}$,
   für alle $\alpha \in \mathsf{Act}$,
   für alle $s_2' \in \mathsf{Proc}$
   wenn $s_2 \xrightarrow{\alpha} s_2'$,
   dann gibt es s_1' mit $s_1 \xrightarrow{\alpha} s_1'$ mit $(s_1', s_2') \in \mathcal{R}$.

Seien $s_1, s_2 \in \mathsf{Proc}$.

- s_1 *und* s_2 *sind (stark) bisimilar*, geschrieben $s_1 \sim s_2$,
  falls es eine (starke) Bisimulation $\mathcal{R}$ gibt mit $(s_1, s_2) \in \mathcal{R}$.

Die Relation $\sim$ heißt *(starke) Bisimulationsäquivalenz / (starke) Bisimilarität*.

[6] Siehe auch Exercise 3.17 [AILS07]

3.7 Theorem (Theorem 3.1 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$ ein beliebiges LTS.
Dann gilt für die Relation $\sim$ auf T:

1. $\sim$ ist eine Äquivalenz.
2. $\sim$ ist die größte (starke) Bisimulation.[7]
3. $\sim$ erfüllt die (starke) Bisimulationsbedingung, d.h.[8]
   $s_1 \sim s_2$ genau dann, wenn für alle $\alpha \in \mathsf{Act}$ gilt:
   - wenn $s_1 \xrightarrow{\alpha} s_1'$, dann gibt es $s_2 \xrightarrow{\alpha} s_2'$ mit $s_1' \sim s_2'$;
   - wenn $s_2 \xrightarrow{\alpha} s_2'$, dann gibt es $s_1 \xrightarrow{\alpha} s_1'$ mit $s_1' \sim s_2'$.

3.8 Theorem (Theorem 3.2 in [AILS07])
Seien P, Q, R CCS-Prozesse.
Sei $P \sim Q$.
Dann gilt

1. für alle $\alpha \in \mathsf{Act}$: $\alpha.P \sim \alpha.Q$;
2. für alle R : $P + R \sim Q + R$ und $R + P \sim R + Q$;
3. für alle R : $P \mid R \sim Q \mid R$ und $R \mid P \sim R \mid Q$;
4. für alle Umbenennungen f : $P[f] \sim Q[f]$;
5. für alle $A \subseteq \mathcal{A}$: $P \setminus A \sim Q \setminus A$.

Schwache Bi-/Simulation

3.9 Definition (Schwache Transition; Definition 3.3 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Seien $s, s' \in \mathsf{Proc}$ und $\alpha \in \mathsf{Act}$.
Wir schreiben $s \overset{\alpha}{\Rightarrow} s'$, falls

1. $\alpha = \tau$ und $s \ (\xrightarrow{\tau})^* \ s'$, oder
2. $\alpha \neq \tau$ und $s \ (\xrightarrow{\tau})^* \ t \xrightarrow{\alpha} t' \ (\xrightarrow{\tau})^* \ s'$ für $t, t' \in \mathsf{Proc}$.

3.10 Definition (Schwache Simulation)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{R} \subseteq \mathsf{Proc} \times \mathsf{Proc}$.
$\mathcal{R}$ heißt *schwache Simulation*, wenn gilt:
für alle $(s_1, s_2) \in \mathcal{R}$,
für alle $\alpha \in \mathsf{Act}$,
für alle $s_1' \in \mathsf{Proc}$
wenn $s_1 \xrightarrow{\alpha} s_1'$,
dann gibt es s_2' mit $s_2 \overset{\alpha}{\Rightarrow} s_2'$ mit $(s_1', s_2') \in \mathcal{R}$.

Seien $s_1, s_2 \in \mathsf{Proc}$.

[7] Vergleiche mit der Relation $\bigcup\{ \mathcal{B} \mid \mathcal{B} \text{ ist Bisimulation } \}$.
[8] im Vergleich zu Definition 3.6 etwas verkürzt ausgedrückt

- s_1 *wird von* s_2 *schwach simuliert*, geschrieben $s_1 \lessapprox s_2$,
  falls es eine schwache Simulation $\mathcal{R}$ gibt mit $(s_1, s_2) \in \mathcal{R}$.

- s_1 *und* s_2 *simulieren sich gegenseitig schwach*, geschrieben $s_1 \approxeq s_2$,
  falls es zwei schwache Simulationen $\mathcal{R}_1$ und $\mathcal{R}_2$ gibt mit

$$(s_1, s_2) \in \mathcal{R}_1 \text{ und } (s_2, s_1) \in \mathcal{R}_2 \ .$$

3.11 Definition (Alternative Definition für schwache Bisimulation)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{R} \subseteq \mathsf{Proc} \times \mathsf{Proc}$.
$\mathcal{R}$ heißt *schwache Bisimulation*,
wenn sowohl $\mathcal{R}$ als auch $\mathcal{R}^{-1}$ eine schwache Simulation ist.

3.12 Definition (Schwache Bisimulation; Definition 3.3 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{R} \subseteq \mathsf{Proc} \times \mathsf{Proc}$.
$\mathcal{R}$ heißt *schwache Bisimulation*, wenn gilt:

1. für alle $(s_1, s_2) \in \mathcal{R}$,
   für alle $\alpha \in \mathsf{Act}$,
   für alle $s_1' \in \mathsf{Proc}$
   wenn $s_1 \xrightarrow{\alpha} s_1'$,
   dann gibt es s_2' mit $s_2 \stackrel{\alpha}{\Rightarrow} s_2'$ mit $(s_1', s_2') \in \mathcal{R}$.

2. für alle $(s_1, s_2) \in \mathcal{R}$,
   für alle $\alpha \in \mathsf{Act}$,
   für alle $s_2' \in \mathsf{Proc}$
   wenn $s_2 \xrightarrow{\alpha} s_2'$,
   dann gibt es s_1' mit $s_1 \stackrel{\alpha}{\Rightarrow} s_1'$ mit $(s_1', s_2') \in \mathcal{R}$.

Seien $s_1, s_2 \in \mathsf{Proc}$.

- s_1 *und* s_2 *sind schwach bisimilar*, geschrieben $s_1 \approx s_2$,
  falls es eine schwache Bisimulation $\mathcal{R}$ gibt mit $(s_1, s_2) \in \mathcal{R}$.

Die Relation $\approx$ heißt *schwache Bisimulationsäquivalenz* bzw. *schwache Bisimilarität*; $\approx$ heißt auch *Beobachtungsäquivalenz*.

3.13 Theorem (Theorem 3.3 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$ ein beliebiges LTS.
Dann gilt für die Relation $\approx$ auf T:

1. $\approx$ ist eine Äquivalenz.

2. $\approx$ ist die größte schwache Bisimulation.

3. $\approx$ erfüllt die schwache Bisimulationsbedingung, d.h.
   $s_1 \approx s_2$ genau dann, wenn für alle $\alpha \in \mathsf{Act}$ gilt:

   - wenn $s_1 \xrightarrow{\alpha} s_1'$, dann gibt es $s_2 \stackrel{\alpha}{\Rightarrow} s_2'$ mit $s_1' \approx s_2'$;
   - wenn $s_2 \xrightarrow{\alpha} s_2'$, dann gibt es $s_1 \stackrel{\alpha}{\Rightarrow} s_1'$ mit $s_1' \approx s_2'$.

3.14 Theorem (Theorem 3.4 in [AILS07])
Seien P, Q, R CCS-Prozesse.
Sei $P \approx Q$.
Dann gilt

1. für alle $\alpha \in \mathsf{Act} : \alpha.P \approx \alpha.Q$;
2. für alle $R : P \mid R \approx Q \mid R$ und $R \mid P \approx R \mid Q$;
3. für alle Umbenennungen $f : P[f] \approx Q[f]$;
4. für alle $A \subseteq \mathcal{A} : P \setminus A \approx Q \setminus A$.

Spieltheorie

3.15 Definition (Spielpositionen)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Dann ist die Menge der *Spielpositionen in* T aufgeteilt in:

- *Angriffspositionen* $G_a^T \triangleq \mathsf{Proc} \times \mathsf{Proc}$ (kurz: G_a),
- *Verteidigungspositionen* $G_d^T \triangleq (\mathsf{Proc} \times \mathsf{Proc}) \times (\mathsf{Act} \times \{l, r\})$ (kurz: G_d).

Wir definieren auch $G \triangleq G_a \cup G_d$.

3.16 Definition (Spielzüge)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Die Menge der *Spielzüge in* T ist definiert
als Relation $\rightarrowtail \subseteq G \times G$ (genauer: $\rightarrowtail \subseteq \big((G_a \times G_d) \cup (G_d \times G_a)\big)$)

1. *Angriff links*: $(p, q) \rightarrowtail ((p', q), (\alpha, l))$,
   falls $(p \xrightarrow{\alpha} p') \in \mathsf{Tran}$.
2. *Angriff rechts*: $(p, q) \rightarrowtail ((p, q'), (\alpha, r))$,
   falls $(q \xrightarrow{\alpha} q') \in \mathsf{Tran}$.
3. l-*Verteidigung*: $((p', q), (\alpha, l)) \rightarrowtail (p', q')$,
   falls $(q \xrightarrow{\alpha} q') \in \mathsf{Tran}$.
4. r-*Verteidigung*: $((p, q'), (\alpha, r)) \rightarrowtail (p', q')$,
   falls $(p \xrightarrow{\alpha} p') \in \mathsf{Tran}$.

3.17 Definition (Spielstand)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Ein *Spielstand* g in T, ausgehend von einer Angriffsposition $g_{\text{init}} \in G_a$,
ist eine Abbildung

$$g : \mathbb{N} \to G \cup \{\bot\}$$

mit

- $g(0) = g_{\text{init}}$ und
- für alle $i > 0$ mit $g(i) \neq \bot$ gilt $g(i{-}1) \rightarrowtail g(i)$.

Die Anzahl z_g der Spielzüge in g ist gegeben durch:

$$z_g \triangleq \begin{cases} \infty & \text{falls } g : \mathbb{N} \to G \\ \max\{\, i \mid g(i) \neq \bot \,\} & \text{sonst} \end{cases}$$

$\mathcal{G}(g_{init})$ bezeichnet die Menge *aller* Spielstände ab g_{init}.

3.18 Definition (Spiel)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Ein *Spiel* in T, ausgehend von $g_{init} \in G_a$,
ist ein *maximaler* Spielstand g in T, also entweder

- eine unendlich lange Sequenz (mit $z_g = \infty$), oder
- eine endliche Sequenz (mit $z_g \in \mathbb{N}$ Spielzügen),
  für die es kein $g \in G$ gibt, so dass $g(z_g) \rightarrowtail g$.

Wir schreiben $\mathcal{G}^{\surd}(g_{init})$ für die Menge aller Spiele ab g_{init}.

3.19 Bemerkung Da Spiel[ständ]e in G_a starten, gilt[9] per Definition 3.16:

- $\forall i \in \mathbb{N}^{[0]} \;.\; g(i) \in (G_a \cup \{\bot\})$
- $\forall i \in \mathbb{N}^{[1]} \;.\; g(i) \in (G_d \cup \{\bot\})$

3.20 Definition (Spielbewertung)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Unendliche Spiele g in T gewinnt die Verteidigung.
Endliche Spiele g in T verliert diejenige Seite,
die in T nicht mehr ziehen kann:

- der Angriff gewinnt, wenn $g(z_g) \in G_d$, und
- die Verteidigung gewinnt, wenn $g(z_g) \in G_a$.

3.21 Bemerkung Spiele werden von genau einer Seite gewonnen.

3.22 Definition (Spielstrategie)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.

- Eine *Angriffsstrategie* ist eine
  Abbildung $f_a : G_a \to (G_d \cup \{\bot\})$ mit
  - $f_a(g) \in G_d$ impliziert $g \rightarrowtail f_a(g)$, und
  - $f_a(g) = \bot$ impliziert, dass es kein $g' \in G_d$ gibt mit $g \rightarrowtail g'$.
- Eine *Verteidigungsstrategie* ist eine
  Abbildung $f_d : G_d \to (G_a \cup \{\bot\})$ mit
  - $f_d(g) \in G_a$ impliziert $g \rightarrowtail f_d(g)$, und
  - $f_d(g) = \bot$ impliziert, dass es kein $g' \in G_a$ gibt mit $g \rightarrowtail g'$.

[9] Wir notieren $\mathbb{N}^{[0]}$ ($\mathbb{N}^{[1]}$) für die Menge der geraden (ungeraden) Elemente von $\mathbb{N}$.

3.23 Definition (Konsistenz)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $g \in \mathcal{G}(g_{\text{init}})$ ein Spielstand in T.
Seien f_a und f_d Strategien wie in Definition 3.22.

- g ist konsistent mit f_a,
  falls für alle $i \in \mathbb{N}^{[0]}$ mit $g(i{+}1) \neq \bot$ gilt: $g(i{+}1) = f_a(g(i))$.
- g ist konsistent mit f_d,
  falls für alle $i \in \mathbb{N}^{[1]}$ mit $g(i{+}1) \neq \bot$ gilt: $g(i{+}1) = f_d(g(i))$.

3.24 Definition (Gewinnstrategie)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$. Sei $g_{\text{init}} \in G_a$.
Eine Spielseite besitzt eine *universelle Gewinnstrategie* für g_{init}, wenn sie alle Spiele in $\mathcal{G}^{\surd}(g_{\text{init}})$, die konsistent mit dieser Strategie sind, gewinnt.

3.25 Bemerkung
Eine universelle Gewinnstrategie für die Verteidigung nennen wir auch *universelle Verteidigungsstrategie*. Eine universelle Gewinnstrategie für den Angriff nennen wir auch *universelle Angriffsstrategie*.

3.26 Proposition (Proposition 3.3 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$. Seien $s_1, s_2 \in \mathsf{Proc}$.
Dann sind s_1 und s_2 genau dann bisimilar, wenn es eine universelle Verteidigungsstrategie für Spiele ab Angriffsposition $g_{\text{init}} = (s_1, s_2)$ gibt.

3.27 Bemerkung
Im Umkehrschluss gilt, dass eine universelle *Angriffsstrategie* dazu benutzt werden kann, um zu zeigen, dass s_1 und s_2 *nicht bisimilar* sind.

3.28 Bemerkung (Proposition 3.4 in [AILS07])
Das Spiel für *schwache* Bisimulation ergibt sich in gleicher Weise wie für starke Bisimulation in den Definitionen 3.15– 3.24, außer dass in der Definition 3.16 die Verteidigung mehr Möglichkeiten bekommt, einen validen Spielzug durchzuführen.

1. *Angriff links*: $(p, q) \rightarrowtail ((p', q), (\alpha, l))$,
   falls $(p \xrightarrow{\alpha} p') \in \mathsf{Tran}$.
2. *Angriff rechts*: $(p, q) \rightarrowtail ((p, q'), (\alpha, r))$,
   falls $(q \xrightarrow{\alpha} q') \in \mathsf{Tran}$.
3. l-*Verteidigung*: $((p', q), (\alpha, l)) \rightarrowtail (p', q')$,
   falls $(q \stackrel{\alpha}{\Rightarrow} q') \in \mathsf{Tran}$.
4. r-*Verteidigung*: $((p, q'), (\alpha, r)) \rightarrowtail (p', q')$,
   falls $(p \stackrel{\alpha}{\Rightarrow} p') \in \mathsf{Tran}$.

Dementsprechend gilt die analoge Aussage zu Proposition 3.26 für die „spielerische“ Entscheidung der schwachen Bisimilarität.

4 Fixpunkttheorie

4.1 Definition (Fixpunkt; Teil von Definition 4.4 in [AILS07])
Sei $f : D \to D$ eine Abbildung.
Sei $d \in D$.
Wenn $f(d) = d$, dann heißt d *Fixpunkt von* f.

4.2 Bemerkung Im Rest dieses Dokuments gehen wir implizit immer davon aus, dass D eine nicht-leere Menge ist.

4.3 Definition (Eigenschaften homogener Relationen; Definition 4.1 in [AILS07])
Sei $R : D \times D$ eine beliebige homogene Relation (auf der *Trägermenge* D).
Dann heißt R:

reflexiv	falls	für alle $a \in D$:	$a \mathrel{R} a$
transitiv	falls	für alle $a, b, c \in D$:	$(a \mathrel{R} b \wedge b \mathrel{R} c) \to a \mathrel{R} c$
antisymmetrisch	falls	für alle $a, b \in D$:	$a \neq b \to (a \mathrel{R} b \to \neg(b \mathrel{R} a))$
linear	falls	für alle $a, b \in D$:	$a \neq b \to (a \mathrel{R} b \vee b \mathrel{R} a)$

- (D, R) heißt *partielle Ordnung*
  bzw. *partiell geordnete Menge* (*poset*),
  falls R reflexiv, transitiv und antisymmetrisch ist.
- Eine partielle Ordnung (D, R) heißt *totale Ordnung*
  bzw. *total geordnete Menge*,
  falls R zudem linear ist.
- Wir benutzen typischerweise $\leqslant$ oder $\sqsubseteq$
  als Metavariable für Ordnungsrelationen.

4.4 Definition (Extremwerte)
Sei $(D, \sqsubseteq)$ eine partielle Ordnung. Sei $A \subseteq D$.

1. (a) $k \in A$ heißt *kleinstes Element von* A, falls gilt:

$$\forall a \in A \;.\; k \sqsubseteq a$$

   (b) $g \in A$ heißt *größtes Element von* A, falls gilt:

$$\forall a \in A \;.\; a \sqsubseteq g$$

   Ein kleinstes Element heißt auch *Minimum von* A,
   ein größtes Element heißt auch *Maximum von* A,
   notiert als $\mathrm{Min}_{\sqsubseteq}(A)$ bzw. $\mathrm{Max}_{\sqsubseteq}(A)$.
   Als Spezialfall notieren wir $\bot \triangleq \mathrm{Min}_{\sqsubseteq}(D)$ und $\top \triangleq \mathrm{Max}_{\sqsubseteq}(D)$.

2. (a) $u \in A$ heißt *minimales Element in* A, falls gilt:

$$\forall a \in A \;.\; \big(a \neq u \;\to\; \neg(a \sqsubseteq u)\big)$$

   (b) $o \in A$ heißt *maximales Element in* A, falls gilt:

$$\forall a \in A \;.\; \big(a \neq o \;\to\; \neg(o \sqsubseteq a)\big)$$

4.5 Bemerkung Bezüglich einer partiellen Ordnung $(D, \sqsubseteq)$ existieren Extremwerte nicht notwendigerweise für jede Teilmenge $A \subseteq D$. Minimale und maximale Elemente sind zudem nicht immer eindeutig.

4.6 Definition (Schranken; Definition 4.2 in [AILS07])
Sei $(D, \sqsubseteq)$ eine partielle Ordnung. Sei $A \subseteq D$.

1. $u \in D$ heißt *untere Schranke* von A (bzgl. D), falls gilt:

$$\forall a \in A \ . \ u \sqsubseteq a$$

Eine *größte untere Schranke von* A *(bzgl.* D*)*, geschrieben $\bigsqcap A$,
auch als *Infimum von* A *(bzgl.* D*)* mit $\inf^D(A)$ bezeichnet,
ist definiert als ein größtes Element
in der Menge der unteren Schranken von A (bzgl. D).

2. $o \in D$ heißt *obere Schranke* von A (bzgl. D), falls gilt:

$$\forall a \in A \ . \ a \sqsubseteq o$$

Eine *kleinste obere Schranke von* A *(bzgl.* D*)*, geschrieben $\bigsqcup A$,
auch als *Supremum von* A *(bzgl.* D*)* mit $\sup^D(A)$ bezeichnet,
ist definiert als ein kleinstes Element
in der Menge der oberen Schranken von A (bzgl. D).

4.7 Bemerkung
Sei $(D, \sqsubseteq)$ eine partielle Ordnung.
Dann können sowohl $\bigsqcap$ als auch $\bigsqcup$
als partielle Funktionen des Typs $2^D \to D$ verstanden werden.

4.8 Bemerkung Bezüglich einer partiellen Ordnung $(D, \sqsubseteq)$ existieren Schranken nicht notwendigerweise für jede Teilmenge $A \subseteq D$.

Schranken werden aus D gewählt. Sie liegen nicht notwendigerweise innerhalb der Menge A, die sie beschränken.

4.9 Bemerkung
Sei $(D, \sqsubseteq)$ eine partielle Ordnung.
Dann gelten $\top = \bigsqcup D = \bigsqcap \emptyset$ und $\bot = \bigsqcap D = \bigsqcup \emptyset$ (wenn diese existieren).

4.10 Lemma
Sei $(D, \sqsubseteq)$ eine totale Ordnung.
Sei $A \subseteq D$ endliche Teilmenge (also $\#(A) \in \mathbb{N}^+$).
Dann sind $\inf^D(A) \in A$ und $\sup^D(A) \in A$.
Somit sind $\mathrm{Min}_{\sqsubseteq}(A)$ und $\mathrm{Max}_{\sqsubseteq}(A)$ definiert.

4.11 Definition (Verband)
Sei $(D, \sqsubseteq)$ eine partielle Ordnung.
Falls für alle $\{ d_1, d_2 \} \subseteq D$
sowohl $\bigsqcap\{ d_1, d_2 \}$ als auch $\bigsqcup\{ d_1, d_2 \}$ existieren,
dann heißt $(D, \sqsubseteq)$ *Verband*.

4.12 Definition (Vollständiger Verband; Definition 4.3 in [AILS07])
Sei $(D, \sqsubseteq)$ eine partielle Ordnung.
Falls für alle $A \subseteq D$
sowohl $\bigsqcap A$ als auch $\bigsqcup A$ existieren,
dann heißt $(D, \sqsubseteq)$ *vollständiger Verband*.

4.13 Lemma Jeder endliche Verband ist vollständig.

4.14 Definition (Monotonie; Teil von Definition 4.4 in [AILS07])
Sei $(D, \sqsubseteq)$ partielle Ordnung.
Sei $f : D \to D$ eine Abbildung.
Dann ist f *monoton bzgl.* $\sqsubseteq$, falls gilt:

für alle $d, d' \in D$,
wenn $d \sqsubseteq d'$, dann auch $f(d) \sqsubseteq f(d')$.

4.15 Definition (Definition 4.5 in [AILS07])
Sei D eine Menge. Sei $f : D \to D$.
Für $n \in \mathbb{N}$ ist f^n induktiv definiert durch:

$$\begin{aligned} f^0(d) &\triangleq d \\ f^{n+1}(d) &\triangleq f(f^n(d)) \end{aligned}$$

4.16 Definition („Fast"-Fixpunkte)
Sei $(D, \sqsubseteq)$ eine partielle Ordnung.
Sei $f : D \to D$ eine Abbildung.
Sei $d \in D$.
Wenn $f(d) \sqsubseteq d$, dann heißt d *Prä-Fixpunkt von* f.
Wenn $d \sqsubseteq f(d)$, dann heißt d *Post-Fixpunkt von* f.

4.17 Theorem (Tarski's Fixpunkttheorem; Theorem 4.1 in [AILS07])
Sei $(D, \sqsubseteq)$ ein vollständiger Verband.
Sei $f : D \to D$ monoton.
Dann gibt es einen größten und einen kleinsten Fixpunkt von f,
bestimmt durch:

$$z_{\max} \triangleq \bigsqcup \{ x \in D \mid x \sqsubseteq f(x) \}$$

$$z_{\min} \triangleq \bigsqcap \{ x \in D \mid f(x) \sqsubseteq x \}$$

4.18 Theorem (Knaster/Tarski; Theorem 4.2 in [AILS07])
Sei $(D, \sqsubseteq)$ ein endlicher vollständiger Verband.
Sei $f : D \to D$ monoton.
Dann wird der größte Fixpunkt von f berechnet durch:

$$z_{\max} \triangleq f^M(\top)$$

für ein $M \in \mathbb{N}$.
Ebenso wird der kleinste Fixpunkt von f berechnet durch:

$$z_{\min} \triangleq f^m(\bot)$$

für ein $m \in \mathbb{N}$.

4.19 Definition
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Seien $p \in \mathsf{Proc}$ und $\alpha \in \mathsf{Act}$.
Dann heißen die Elemente von

$$\mathrm{Der}(p, \alpha) \triangleq \left\{ p' \in \mathsf{Proc} \mid p \xrightarrow{\alpha} p' \right\}$$

*α-Nachfolger (*bzw. *α-Derivatives) von* p.

4.20 Definition (Alternative Definition für Bisimulation)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{R} : \mathsf{Proc} \times \mathsf{Proc}$.
$\mathcal{R}$ heißt *(starke) Bisimulation*, wenn gilt:

1. $\forall (p, q) \in \mathcal{R}. \forall \alpha \in \mathsf{Act}. \forall p' \in \mathrm{Der}(p, \alpha).$
   $\exists q' \in \mathrm{Der}(q, \alpha). (p', q') \in \mathcal{R}$

2. $\forall (p, q) \in \mathcal{R}. \forall \alpha \in \mathsf{Act}. \forall q' \in \mathrm{Der}(q, \alpha).$
   $\exists p' \in \mathrm{Der}(p, \alpha). (p', q') \in \mathcal{R}$

4.21 Definition (1-Schritt-Bisimulation)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{R} : \mathsf{Proc} \times \mathsf{Proc}$.

„vorwärts“ Die Relation $\overset{1}{\sim}_{\mathcal{R}} : \mathsf{Proc} \times \mathsf{Proc}$ ist gegeben durch:

- $p \overset{1}{\sim}_{\mathcal{R}} q$ gdw.
  1. $\forall \alpha \in \mathsf{Act}. \forall p' \in \mathrm{Der}(p, \alpha). \exists q' \in \mathrm{Der}(q, \alpha). (p', q') \in \mathcal{R}$
  2. $\forall \alpha \in \mathsf{Act}. \forall q' \in \mathrm{Der}(q, \alpha). \exists p' \in \mathrm{Der}(p, \alpha). (p', q') \in \mathcal{R}$

„rückwärts“ Die Funktion $\mathcal{F} : 2^{\mathsf{Proc}\times\mathsf{Proc}} \to 2^{\mathsf{Proc}\times\mathsf{Proc}}$ ist gegeben durch:

- $$\begin{aligned} \mathcal{F}(\mathcal{R}) &\triangleq \overset{1}{\sim}_{\mathcal{R}} \\ &= \left\{ (p,\ q) \in \mathsf{Proc} \times \mathsf{Proc} \mid p \overset{1}{\sim}_{\mathcal{R}} q \right\} \end{aligned}$$

4.22 Lemma
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$. Dann gilt:

1. $\mathcal{F}$ ist monoton bzgl. $\subseteq$.
2. $\mathcal{R}$ ist (starke) Bisimulation gdw. $\mathcal{R} \subseteq \mathcal{F}(\mathcal{R})$.

4.23 Bemerkung
Mit der Beobachtung, dass

$$\begin{aligned} \sim &= \bigcup \left\{ \mathcal{R} \in 2^{(\mathsf{Proc}\times\mathsf{Proc})} \mid \mathcal{R} \text{ ist (starke) Bisimulation} \right\} \\ &= \bigcup \left\{ \mathcal{R} \in 2^{(\mathsf{Proc}\times\mathsf{Proc})} \mid \mathcal{R} \subseteq \mathcal{F}(\mathcal{R}) \right\} \end{aligned}$$

folgt mit Theorem 4.17,
dass $\sim$ genau dem größten Fixpunkt von $\mathcal{F}$ entspricht.

Demzufolge kann $\sim$ für endliche LTS durch Theorem 4.18 als

$$\sim \ = \ \mathcal{F}^{M}(\mathsf{Proc} \times \mathsf{Proc})$$

im Verband $\left(2^{\mathsf{Proc}\times\mathsf{Proc}}, \subseteq\right)$ immer algorithmisch berechnet werden.
Mit $\sim_i \triangleq \mathcal{F}^i(\mathsf{Proc} \times \mathsf{Proc})$ für alle $i \in \mathbb{N}$ gilt zudem[10]:

1. $\sim_i$ ist eine Äquivalenz;
2. $\sim_{i+1} \subseteq \sim_i$.

[10] Vergleiche mit Exercise 4.14 in [AILS07].

5 Hennessy-Milner-Logik

5.1 Definition (HML Syntax; Definition 5.1 in [AILS07])
Die Menge $\mathcal{M}$ der HML-Formeln über Act ist gegegeben durch:

$$F ::= tt \mid ff \mid F \wedge F \mid F \vee F \mid [\alpha]F \mid \langle\alpha\rangle F$$

wobei $\alpha \in \mathsf{Act}$.

Es gilt Operatorenvorrang in absteigender Folge:

- Gleichberechtigt die Modaloperatoren $\langle\alpha\rangle$ und $[\alpha]$ und
- gleichberechtigt die Disjunktion $F \vee F$ und Konjunktion $F \wedge F$.

Alternative: Die Syntax kann auch definiert werden durch:

$$F ::= \bigwedge_{i\in I} F \mid \bigvee_{i\in I} F \mid [\alpha]F \mid \langle\alpha\rangle F$$

wobei $\alpha \in \mathsf{Act}$ und $\#(I) \in \mathbb{N}$.
Dann ergeben sich

$$tt \triangleq \bigwedge_{i\in\emptyset} F_i \qquad F_1 \wedge F_2 \triangleq \bigwedge_{i\in\{1,2\}} F_i$$

$$ff \triangleq \bigvee_{i\in\emptyset} F_i \qquad F_1 \vee F_2 \triangleq \bigvee_{i\in\{1,2\}} F_i$$

als Spezialfälle.[11]
Wir definieren zudem, für $A \subseteq \mathsf{Act}$ mit $\#(A) \in \mathbb{N}$ und F beliebig:

- $[A]F \triangleq \bigwedge_{\alpha\in A} [\alpha]F$
- $\langle A\rangle F \triangleq \bigvee_{\alpha\in A} \langle\alpha\rangle F$

mit $[\emptyset]F = tt$ und $\langle\emptyset\rangle F = ff$.

5.2 Definition (Modaltiefe; S. 126 in [AILS07])
Die *Modaltiefe* $\mathrm{md}(F)$ einer HML-Formel F bezeichnet die maximale Verschachtelungstiefe von Modaloperatoren in F, gegeben durch die Funktion $\mathrm{md} : \mathcal{M} \to \mathbb{N}$ mit:

$$\begin{aligned}
\mathrm{md}(tt) &\triangleq 0\\
\mathrm{md}(ff) &\triangleq 0\\
\mathrm{md}(F_1 \wedge F_2) &\triangleq \max\{\,\mathrm{md}(F_1),\ \mathrm{md}(F_2)\,\}\\
\mathrm{md}(F_1 \vee F_2) &\triangleq \max\{\,\mathrm{md}(F_1),\ \mathrm{md}(F_2)\,\}\\
\mathrm{md}([\alpha]F) &\triangleq 1 + \mathrm{md}(F)\\
\mathrm{md}(\langle\alpha\rangle F) &\triangleq 1 + \mathrm{md}(F)
\end{aligned}$$

[11] Die alternative Syntax kann man in etwa als ein Repräsentantensystem verstehen, das verschachtelte binäre Konjunktionen und Disjunktionen jeweils bis auf Kommutivität, Assoziativität und Neutralität als äquivalent ansieht (vergl. *kommutative Monoide*).

5.3 Definition („Model-Checking-Semantik“ von HML; S. 96 in [AILS07])

Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}$ definiert über Act.
Die Relation $\models : (\mathsf{Proc}, \mathcal{M})$ ist induktiv definiert durch:

$$\begin{array}{lll} p & \models & \mathit{tt} \quad \text{für alle } p \in \mathsf{Proc} \\ p & \models & \mathit{ff} \quad \text{für kein } p \in \mathsf{Proc} \\ p & \models & F \wedge G \quad \text{falls } p \models F \text{ und } p \models G \\ p & \models & F \vee G \quad \text{falls } p \models F \text{ oder } p \models G \\ p & \models & [\alpha]F \quad \text{falls für alle } p' \in \mathrm{Der}(p, \alpha) \text{ gilt } p' \models F \\ p & \models & \langle\alpha\rangle F \quad \text{falls es } p' \in \mathrm{Der}(p, \alpha) \text{ gibt mit } p' \models F \end{array}$$

5.4 Definition (Denotationelle HML-Semantik; Def. 5.2 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}$ definiert über Act.
Die Funktion $[\![\cdot]\!] : \mathcal{M} \to 2^{\mathsf{Proc}}$ ist induktiv definiert durch:

$$\begin{array}{rcl} [\![\mathit{tt}]\!] & \triangleq & \mathsf{Proc} \\ [\![\mathit{ff}]\!] & \triangleq & \emptyset \\ [\![F \wedge G]\!] & \triangleq & [\![F]\!] \cap [\![G]\!] \\ [\![F \vee G]\!] & \triangleq & [\![F]\!] \cup [\![G]\!] \\ [\![[\alpha]F]\!] & \triangleq & [\cdot\alpha\cdot][\![F]\!] \\ [\![\langle\alpha\rangle F]\!] & \triangleq & \langle\cdot\alpha\cdot\rangle[\![F]\!] \end{array}$$

wobei die Operatoren $[\cdot\alpha\cdot], \langle\cdot\alpha\cdot\rangle : 2^{\mathsf{Proc}} \to 2^{\mathsf{Proc}}$ gegeben sind durch:

$$\begin{array}{rcl} [\cdot\alpha\cdot]S & \triangleq & \{\, p \in \mathsf{Proc} \mid \forall p' \in \mathrm{Der}(p,\ \alpha)\ .\ p' \in S \,\} \\ \langle\cdot\alpha\cdot\rangle S & \triangleq & \{\, p \in \mathsf{Proc} \mid \exists p' \in \mathrm{Der}(p,\ \alpha)\ .\ p' \in S \,\} \end{array}$$

Die Semantik $[\![\cdot]\!]$ für die n-stelligen Varianten $\bigwedge$ und $\bigvee$ ergibt sich analog durch die n-stelligen Varianten $\bigcap$ und $\bigcup$.

5.5 Lemma Es gilt:

$$p \models F \quad \text{genau dann, wenn} \quad p \in [\![F]\!]$$

5.6 Definition
Im Kontext von Definition 5.4 benutzen wir[12] auch
$[\cdot A\cdot], \langle\cdot A\cdot\rangle : 2^{\mathsf{Proc}} \to 2^{\mathsf{Proc}}$ für $A \subseteq \mathsf{Act}$ mit $\#(A) \in \mathbb{N}$,
definiert durch

$$[\cdot A\cdot]S \triangleq \bigcap_{\alpha\in A} [\cdot\alpha\cdot]S \qquad \langle\cdot A\cdot\rangle S \triangleq \bigcup_{\alpha\in A} \langle\cdot\alpha\cdot\rangle S$$

zur Definition der HML-Semantik für die alternative Syntax:

$$[A]F \triangleq [\cdot A\cdot][\![F]\!] \qquad \langle A\rangle F \triangleq \langle\cdot A\cdot\rangle[\![F]\!]$$

[12] Dies dient alleine der besseren Lesbarkeit in Beispielen.

5.7 Definition (Negation)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}$ definiert über Act.
Die Funktion $(\cdot)^c : \mathcal{M} \to \mathcal{M}$ ist gegeben durch:

$$\begin{aligned}
(tt)^c &\triangleq ff \\
(ff)^c &\triangleq tt \\
(F \wedge G)^c &\triangleq (F)^c \vee (G)^c \\
(F \vee G)^c &\triangleq (F)^c \wedge (G)^c \\
([\alpha]F)^c &\triangleq \langle\alpha\rangle (F)^c \\
(\langle\alpha\rangle F)^c &\triangleq [\alpha] (F)^c
\end{aligned}$$

Es gilt für alle $F \in \mathcal{M}$:

1. $[\![(F)^c]\!] = \mathsf{Proc} \setminus [\![F]\!]$;
2. $\big((F)^c\big)^c = F$.

5.8 Definition
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}$ definiert über Act.
Die Funktion $[\![\cdot]\!] : \mathsf{Proc} \to 2^{\mathcal{M}}$ ist gegeben durch[13]:

$$[\![p]\!] \triangleq \{ F \in \mathcal{M} \mid p \models F \}$$

Die Funktion $[\![\cdot]\!]^{\leqslant i} : \mathsf{Proc} \to 2^{\mathcal{M}}$ ist gegeben durch:

$$[\![p]\!]^{\leqslant i} \triangleq \{ F \in \mathcal{M} \mid p \models F \wedge \mathrm{md}(F) \leqslant i \}$$

5.9 Definition (Bild-Endlichkeit; Definition 5.3 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.

- $p \in \mathsf{Proc}$ heißt *Bild-endlich*,
  falls für alle $\alpha \in \mathsf{Act}$: $\#(\mathrm{Der}(p, \alpha)) \in \mathbb{N}$.
- T heißt *Bild-endlich*,
  falls alle $p \in \mathsf{Proc}$ Bild-endlich sind.

5.10 Theorem (Hennessy-Milner-Theorem; Theorem 5.1 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$ Bild-endlich.
Seien $p, q \in \mathsf{Proc}$.
Dann gilt:

$p \sim q$ genau dann, wenn $[\![p]\!] = [\![q]\!]$

$p \sim_i q$ genau dann, wenn $[\![p]\!]^{\leqslant i} = [\![q]\!]^{\leqslant i}$

[13] per Overloading, siehe Def. 5.4.

6 HML mit Rekursion

In den Definitionen 5.4 und 5.6 haben wir einige Varianten für die alternative Syntax aus Definition 5.1 explizit angegeben. In diesem Kapitel werden dies implizit lassen, um die Darstellung übersichtlich zu halten. Als einzige Ausnahme gilt Definition 6.22. Daneben benutzen wir die bequem abkürzende Syntax in den Definitionen 6.10 und 6.18.

6.1 Definition
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\alpha \in \mathsf{Act}$.
Sei $A \subseteq \mathsf{Act}$ mit $\#(A) \in \mathbb{N}$.

$$
\begin{array}{lcl}
[\alpha]^{n+1}F & \triangleq & [\alpha]\,([\alpha]^{n}F) \\
[\alpha]^{0}F & \triangleq & F
\end{array}
$$

$$
\begin{array}{lcl}
\langle\alpha\rangle^{n+1}F & \triangleq & \langle\alpha\rangle\,(\langle\alpha\rangle^{n}F) \\
\langle\alpha\rangle^{0}F & \triangleq & F
\end{array}
$$

$$
\begin{array}{lcl}
[A]^{n+1}F & \triangleq & [A]\,([A]^{n}F) \\
[A]^{0}F & \triangleq & F
\end{array}
$$

$$
\begin{array}{lcl}
\langle A\rangle^{n+1}F & \triangleq & \langle A\rangle\,(\langle A\rangle^{n}F) \\
\langle A\rangle^{0}F & \triangleq & F
\end{array}
$$

Rekursion mit einer Variablen

6.2 Definition (1HML Syntax)
Die Menge $\mathcal{M}_{\{X\}}$ der 1HML-Formeln über Act mit einer Variablen X, ist gegegeben durch:

$$F \;::=\; X \;\mid\; tt \;\mid\; ff \;\mid\; F \wedge F \;\mid\; F \vee F \;\mid\; [\alpha]F \;\mid\; \langle\alpha\rangle F$$

wobei $\alpha \in \mathsf{Act}$.

6.3 Bemerkung (1HML Rekursion: Syntaktisch)
Rekursives Verhalten wird durch Gleichungen der Form

$$X \;=\; F_X$$

angegeben, wobei F_X eine 1HML-Formel bezeichnet, in der X vorkommen kann.

Unter der Annahme, dass Fixpunkte solcher Gleichungen existieren, spezifizieren wir die Art des gewünschten Fixpunktes durch

$$X \overset{\max}{=} F_X \qquad \text{und} \qquad X \overset{\min}{=} F_X.$$

6.4 Bemerkung (1HML Rekursion: Semantisch)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}_{\{X\}}$ definiert über Act.
Wenn X durch eine Gleichung der Form

$$X \;=\; F_X$$

spezifiziert ist, dann entspricht dies semantisch (in 2^{Proc}) der Gleichung

$$[\![X]\!] \;=\; \mathcal{O}_{F_X}([\![X]\!])$$

bezüglich einer Funktion

$$\mathcal{O}_{F_X} : 2^{\mathsf{Proc}} \to 2^{\mathsf{Proc}}$$

(als Instanz von $\mathcal{O} : \mathcal{M}_{\{X\}} \to (2^{\mathsf{Proc}} \to 2^{\mathsf{Proc}})$).

Wenn $\mathcal{O}_{F_X}$ monoton bzgl. $\left(2^{\mathsf{Proc}}, \subseteq\right)$ ist, dann
gibt es nach Tarski sowohl eine größte als auch eine kleinste Lösung:

$$\begin{aligned} \mathrm{FIX}\,\mathcal{O}_{F_X} &\triangleq \bigcup \left\{\, S \in 2^{\mathsf{Proc}} \mid S \subseteq \mathcal{O}_{F_X}(S) \,\right\} \\ \mathrm{fix}\,\mathcal{O}_{F_X} &\triangleq \bigcap \left\{\, S \in 2^{\mathsf{Proc}} \mid \mathcal{O}_{F_X}(S) \subseteq S \,\right\} \end{aligned}$$

6.5 Definition (Semantik von 1HML-Rekursion; Definition 6.1 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}_{\{X\}}$ definiert über Act.
Sei $F \in \mathcal{M}_{\{X\}}$.
Die Funktion $\mathcal{O}_F : 2^{\mathsf{Proc}} \to 2^{\mathsf{Proc}}$ ist induktiv definiert durch:

$$\begin{aligned} \mathcal{O}_X(S) &\triangleq S \\ \mathcal{O}_{tt}(S) &\triangleq \mathsf{Proc} \\ \mathcal{O}_{ff}(S) &\triangleq \emptyset \\ \mathcal{O}_{F \wedge G}(S) &\triangleq \mathcal{O}_F(S) \cap \mathcal{O}_G(S) \\ \mathcal{O}_{F \vee G}(S) &\triangleq \mathcal{O}_F(S) \cup \mathcal{O}_G(S) \\ \mathcal{O}_{[\alpha]F}(S) &\triangleq [\cdot\alpha\cdot]\mathcal{O}_F(S) \\ \mathcal{O}_{\langle\alpha\rangle F}(S) &\triangleq \langle\cdot\alpha\cdot\rangle\mathcal{O}_F(S) \end{aligned}$$

6.6 Lemma
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}_{\{X\}}$ definiert über Act.
Dann gilt für alle $F \in \mathcal{M}_{\{X\}}$, dass $\mathcal{O}_F$ monoton bzgl. $\left(2^{\mathsf{Proc}}, \subseteq\right)$ ist.

6.7 Theorem (Theorem 6.1 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}_{\{X\}}$ definiert über Act.
Sei Proc endlich, d.h., $\#(\mathsf{Proc}) \in \mathbb{N}$.
Sei $F_X \in \mathcal{M}_{\{X\}}$.
Dann gilt (mit Theorem 4.18) für eine Gleichung $X = F_X$, dass

$$\begin{aligned} \mathrm{FIX}\,\mathcal{O}_{F_X} &= (\mathcal{O}_{F_X})^{M}(\mathsf{Proc}) \\ \mathrm{fix}\,\mathcal{O}_{F_X} &= (\mathcal{O}_{F_X})^{m}(\emptyset) \end{aligned}$$

für $m, M \in \mathbb{N}$.

6.8 Definition (Denotationelle Semantik von 1HML)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}_{\{X\}}$ definiert über Act.
Die Funktion $[\![\cdot]\!] : \mathcal{M}_{\{X\}} \to 2^{\mathsf{Proc}}$ ist induktiv definiert durch:

$$
\begin{aligned}
[\![tt]\!] &\triangleq \mathsf{Proc} \\
[\![ff]\!] &\triangleq \emptyset \\
[\![F \wedge G]\!] &\triangleq [\![F]\!] \cap [\![G]\!] \\
[\![F \vee G]\!] &\triangleq [\![F]\!] \cup [\![G]\!] \\
[\![[\alpha]F]\!] &\triangleq [\cdot\alpha\cdot][\![F]\!] \\
[\![\langle\alpha\rangle F]\!] &\triangleq \langle\cdot\alpha\cdot\rangle[\![F]\!] \\
[\![X]\!] &\triangleq \begin{cases} \mathrm{FIX}\, \mathcal{O}_{F_X} & \text{falls } X \overset{\max}{=} F_X \\ \mathrm{fix}\, \mathcal{O}_{F_X} & \text{falls } X \overset{\min}{=} F_X \end{cases}
\end{aligned}
$$

„Syntaktischer Zucker“

6.9 Definition
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Es gibt zwei Arten *vollständiger Transitionssequenzen*:

- endliche Sequenzen
  $p_0 \xrightarrow{\alpha_1} p_1 \xrightarrow{\alpha_2} \ldots \xrightarrow{\alpha_n} p_n$ mit $p_n \not\to$, und
- unendliche Sequenzen
  $p_0 \xrightarrow{\alpha_1} p_1 \xrightarrow{\alpha_2} \ldots \xrightarrow{\alpha_n} p_n \xrightarrow{\alpha_{n+1}} \ldots$

6.10 Definition
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}_{\{X\}}$ definiert über Act.
Häufig benutzte Formelschemata sind gegeben wie folgt:

$$
\begin{aligned}
\mathit{Inv}(F) &: \; X \overset{\max}{=} F \wedge [\mathsf{Act}]X \\
\mathit{Pos}(F) &: \; X \overset{\min}{=} F \vee \langle\mathsf{Act}\rangle X \\
\mathit{Safe}(F) &: \; X \overset{\max}{=} F \wedge ([\mathsf{Act}]ff \vee \langle\mathsf{Act}\rangle X) \\
\mathit{Even}(F) &: \; X \overset{\min}{=} F \vee (\langle\mathsf{Act}\rangle tt \wedge [\mathsf{Act}]X) \\
G\ \mathcal{U}^w\ F &: \; X \overset{\max}{=} F \vee (G \wedge [\mathsf{Act}]X) \\
G\ \mathcal{U}^s\ F &: \; X \overset{\min}{=} F \vee (G \wedge [\mathsf{Act}]X \wedge \langle\mathsf{Act}\rangle tt)
\end{aligned}
$$

Mehrere Variablen

6.11 Definition (HML Syntax, mit mehreren (Rekursions-)Variablen)
Sei $\mathcal{X} = \{ X_1, \ldots, X_n \}$ eine nicht-leere Menge von Variablen.
Die Menge $\mathcal{M}_{\mathcal{X}}$ der HML-Formeln über Act mit Variablen in $\mathcal{X}$,
ist gegegeben durch:

$$F \ ::= \ X \mid tt \mid ff \mid F \wedge F \mid F \vee F \mid [\alpha]F \mid \langle\alpha\rangle F$$

wobei $\alpha \in \mathsf{Act}$ und $X \in \mathcal{X}$.

6.12 Bemerkung (Gleichungssysteme: Syntax)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{X} = \{ X_1, \ldots, X_n \}$ eine nicht-leere Menge von Variablen.
Ein *gegenseitig rekursives Gleichungssystem* hat die Form

$$\begin{aligned} X_1 &= F_{X_1} \\ &\vdots \\ X_n &= F_{X_n} \end{aligned}$$

wobei $F_{X_i} \in \mathcal{M}_{\mathcal{X}}$ für $i \in [1, n]$.
Es wird abkürzend beschrieben durch eine *Deklaration*

$$\begin{aligned} \boldsymbol{F} : \mathcal{X} &\to \mathcal{M}_{\mathcal{X}} \\ X &\mapsto F_X \end{aligned}$$

und wir gehen zunächst davon aus, dass wir
bei allen Einzelgleichungen den gleichen Fixpunkt intendieren.

6.13 Definition (HML Semantik, mit mehreren (Rekursions-)Variablen)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{X} = \{ X_1, \ldots, X_n \}$ eine nicht-leere Menge von Variablen.
Die Funktion $\mathcal{O}_F : (2^{\mathsf{Proc}})^n \to 2^{\mathsf{Proc}}$ ist induktiv definiert durch:

$$\begin{aligned} \mathcal{O}_{X_i}(S_1, \ldots, S_n) &\triangleq S_i \\ \mathcal{O}_{tt}(S_1, \ldots, S_n) &\triangleq \mathsf{Proc} \\ \mathcal{O}_{ff}(S_1, \ldots, S_n) &\triangleq \emptyset \\ \mathcal{O}_{F \wedge G}(S_1, \ldots, S_n) &\triangleq \mathcal{O}_F(S_1, \ldots, S_n) \cap \mathcal{O}_G(S_1, \ldots, S_n) \\ \mathcal{O}_{F \vee G}(S_1, \ldots, S_n) &\triangleq \mathcal{O}_F(S_1, \ldots, S_n) \cup \mathcal{O}_G(S_1, \ldots, S_n) \\ \mathcal{O}_{[\alpha]F}(S_1, \ldots, S_n) &\triangleq [\cdot\alpha\cdot]\mathcal{O}_F(S_1, \ldots, S_n) \\ \mathcal{O}_{\langle\alpha\rangle F}(S_1, \ldots, S_n) &\triangleq \langle\cdot\alpha\cdot\rangle\mathcal{O}_F(S_1, \ldots, S_n) \end{aligned}$$

6.14 Bemerkung (Gleichungssysteme: Semantik)
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{X} = \{X_1, \ldots, X_n\}$ eine nicht-leere Menge von Variablen.
Das syntaktische Gleichungssystem aus 6.12
entspricht dem semantischen Gleichungssystem

$$\begin{aligned} S_1 &= \mathcal{O}_{F_{X_1}}(S_1, \ldots, S_n) \\ &\vdots \\ S_n &= \mathcal{O}_{F_{X_n}}(S_1, \ldots, S_n) \end{aligned}$$

mit $S_i \in 2^{\mathsf{Proc}}$.

6.15 Lemma
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{D} \triangleq (2^{\mathsf{Proc}})^n$ für $n \in \mathbb{N}$.
Sei $\sqsubseteq : (\mathcal{D}, \mathcal{D})$ definiert durch

$$\begin{pmatrix} S_1 \\ \vdots \\ S_n \end{pmatrix} \sqsubseteq \begin{pmatrix} S_1' \\ \vdots \\ S_n' \end{pmatrix} \quad \text{falls } S_i \subseteq S_i' \text{ für alle } i \in [1, n].$$

Dann ist $(\mathcal{D}, \sqsubseteq)$ ist ein vollständiger Verband, mit

$$\bigsqcup \left\{ \begin{pmatrix} S_1^i \\ \vdots \\ S_n^i \end{pmatrix} \mid i \in I \right\} \triangleq \begin{pmatrix} \bigcup_{i \in I} S_1^i \\ \vdots \\ \bigcup_{i \in I} S_n^i \end{pmatrix}$$

$$\bigsqcap \left\{ \begin{pmatrix} S_1^i \\ \vdots \\ S_n^i \end{pmatrix} \mid i \in I \right\} \triangleq \begin{pmatrix} \bigcap_{i \in I} S_1^i \\ \vdots \\ \bigcap_{i \in I} S_n^i \end{pmatrix}$$

wobei I eine beliebige Indexmenge ist.

6.16 Lemma
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $\mathcal{X} = \{X_1, \ldots, X_n\}$ eine nicht-leere Menge von Variablen.
Sei $\mathcal{D} = (2^{\mathsf{Proc}})^n$.
Sei $\boldsymbol{F} : \mathcal{X} \to \mathcal{M}_{\mathcal{X}}$ eine Deklaration.
Dann ist $\mathcal{O}_{\boldsymbol{F}} : \mathcal{D} \to \mathcal{D}$ gegeben durch

$$\begin{pmatrix} S_1 \\ \vdots \\ S_n \end{pmatrix} \mapsto \begin{pmatrix} \mathcal{O}_{\boldsymbol{F}(X_1)}(S_1, \ldots, S_n) \\ \vdots \\ \mathcal{O}_{\boldsymbol{F}(X_n)}(S_1, \ldots, S_n) \end{pmatrix}$$

eine monotone Funktion auf $(\mathcal{D}, \sqsubseteq)$.

6.17 Definition (Denotationelle Semantik von HML)
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$ endliches LTS.
Sei $\mathcal{X} = \{ X_1, \ldots, X_n \}$ eine nicht-leere Menge von Variablen.
Sei $\boldsymbol{F} : \mathcal{X} \to \mathcal{M}_{\mathcal{X}}$ eine Deklaration
mit $X_i \overset{m}{=} \boldsymbol{F}(X_i)$ für alle $1 \leqslant i \leqslant n$ und $m \in \{\max, \min\}$.
Die Funktion $[\![\cdot]\!] : \mathcal{M}_{\mathcal{X}} \to 2^{\mathsf{Proc}}$ ist induktiv definiert durch:

$$
\begin{aligned}
[\![\mathit{tt}]\!] &\triangleq \mathsf{Proc} \\
[\![\mathit{ff}]\!] &\triangleq \emptyset \\
[\![F \wedge G]\!] &\triangleq [\![F]\!] \cap [\![G]\!] \\
[\![F \vee G]\!] &\triangleq [\![F]\!] \cup [\![G]\!] \\
[\![[\alpha] F]\!] &\triangleq [\cdot \alpha \cdot] [\![F]\!] \\
[\![\langle \alpha \rangle F]\!] &\triangleq \langle \cdot \alpha \cdot \rangle [\![F]\!] \\
[\![X_i]\!] &\triangleq \begin{cases} (\mathrm{FIX}\, \mathcal{O}_{\boldsymbol{F}})_i & \text{falls } m = \max \\ (\mathrm{fix}\, \mathcal{O}_{\boldsymbol{F}})_i & \text{falls } m = \min \end{cases}
\end{aligned}
$$

wobei FIX und fix (modulo Typ) analog zu Theorem 6.7 gegeben sind.

6.18 Definition (Charakteristische Deklaration; Teil von Theorem 6.4 in [AILS07])
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$ ein endliches LTS.
Sei $\mathcal{X}_{\mathsf{T}} \triangleq \{ X_p \mid p \in \mathsf{Proc} \}$ eine Menge von Variablen,
deren Bedeutung wir durch Rekursionsgleichungen bestimmen.
Die *charakteristische Deklaration* ist hierzu gegeben durch:

$$
\begin{aligned}
\boldsymbol{F}_{\mathsf{T}} : \mathcal{X}_{\mathsf{T}} &\to \mathcal{M}_{\mathcal{X}_{\mathsf{T}}} \\
X_p &\mapsto \bigwedge_{\alpha \in \mathsf{Act}} \Big(\bigwedge_{p' \in \mathrm{Der}(p,\alpha)} \langle \alpha \rangle X_{p'} \Big) \wedge \bigwedge_{\alpha \in \mathsf{Act}} [\alpha] \Big(\bigvee_{p' \in \mathrm{Der}(p,\alpha)} X_{p'} \Big)
\end{aligned}
$$

6.19 Theorem (Charakteristische Formeln; Teil von Theorem 6.4 in [AILS07])
Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$ ein endliches LTS.
Sei $\boldsymbol{F}_{\mathsf{T}}$ die charakteristische Deklaration, wie in 6.18 definiert.
Sei

$$X_p \overset{\max}{=} \boldsymbol{F}_{\mathsf{T}}(X_p)$$

für alle $p \in \mathsf{Proc}$.
Dann gilt[14]

$$[\![X_p]\!] = [p]_{\sim}$$

weshalb X_p jeweils die *charakteristische Formel für* p genannt wird.

[14] Im Vergleich zu [AILS07] wählen wir hier die Variante aus Lemma 6.3, da sie (ohne Verweis auf andere Prozesse) die charakterische Eigenschaft expliziter widerspiegelt.

6.20 Definition (Min-Max-Mix)
Sei $\mathcal{X} \triangleq (\mathcal{X}_k)_{k\in[1,n]}$ eine Folge von Mengen $\mathcal{X}_k \triangleq \{X_{k,1}, \ldots, X_{k,z_k}\}$,
die jeweils z_k Variablen enthalten.

Sei $\mathcal{X}^{\leqslant i} \triangleq \{ X_{k,l} \mid k\in[1,i] \text{ und } l\in[1,z_k] \}$ die *Menge* der Variablen in $\mathcal{X}^{\leqslant i}$.

Sei $(\boldsymbol{F}_i)_{i\in[1,n]}$ eine Folge von Deklarationen mit $\boldsymbol{F}_i : \mathcal{X}_i \to \mathcal{M}_{\mathcal{X}^{\leqslant i}}$.

Sei $(m_i)_{i\in[1,n]}$ eine Folge von Fixpunkt-Indikatoren
mit $m_i \in \{\max, \min\}$ und $m_i \neq m_{i+1}$ für alle $i \in [1, n-1]$.

Dann heißt $\langle(\boldsymbol{F}_1, \mathcal{X}_1, m_1), \cdots, (\boldsymbol{F}_n, \mathcal{X}_n, m_n)\rangle$
n-verschachteltes gegenseitig rekursives Gleichungssystem,
wobei $(\boldsymbol{F}_i, \mathcal{X}_i, m_i)$ i-ter Block genannt wird.

Sei $\mathsf{T} = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.

Die partiellen Funktionen $\mathbf{f}_i : [1,n] \times \mathbb{N} \rightharpoonup 2^{\mathsf{Proc}}$ für $i \in [1,n]$ sind mit Bezug auf die Abbildung $[\![\cdot]\!] : \mathcal{M}_{\mathcal{X}^{\leqslant n}} \to 2^{\mathsf{Proc}}$ (siehe unten) definiert als

$$\mathbf{f}_i(k,j) \triangleq [\![X_{k,j}]\!]$$

falls $k \in [1, i-1]$ und $j \in [1, z_k]$, ansonsten undefiniert.

Die Funktionen $\mathcal{O}_{\boldsymbol{F}_i(X_{i,j});\mathbf{f}_i} : (2^{\mathsf{Proc}})^{z_i} \to 2^{\mathsf{Proc}}$ für $j \in [1, z_i]$
sind induktiv definiert durch:

$$\begin{aligned}
\mathcal{O}_{X_{k,l};\mathbf{f}_i}(S_1,\ldots,S_{z_i}) &\triangleq \begin{cases} S_l & \text{falls } k = i \\ \mathbf{f}_i(k,l) & \text{andernfalls} \end{cases} \\
\mathcal{O}_{tt;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) &\triangleq \mathsf{Proc} \\
\mathcal{O}_{ff;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) &\triangleq \emptyset \\
\mathcal{O}_{F\wedge G;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) &\triangleq \mathcal{O}_{F;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) \cap \mathcal{O}_{G;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) \\
\mathcal{O}_{F\vee G;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) &\triangleq \mathcal{O}_{F;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) \cup \mathcal{O}_{G;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) \\
\mathcal{O}_{[\alpha]F;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) &\triangleq [\cdot\alpha\cdot]\mathcal{O}_{F;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) \\
\mathcal{O}_{\langle\alpha\rangle F;\mathbf{f}_i}(S_1,\ldots,S_{z_i}) &\triangleq \langle\cdot\alpha\cdot\rangle\mathcal{O}_{F;\mathbf{f}_i}(S_1,\ldots,S_{z_i})
\end{aligned}$$

und wie in Lemma 6.16 geliftet auf $\mathcal{O}_{\boldsymbol{F}_i;\mathbf{f}_i} : (2^{\mathsf{Proc}})^{z_i} \to (2^{\mathsf{Proc}})^{z_i}$.

Die Funktion $[\![\cdot]\!] : \mathcal{M}_{\mathcal{X}^{\leqslant n}} \to 2^{\mathsf{Proc}}$ ist schließlich induktiv definiert durch:

$$\begin{aligned}
[\![tt]\!] &\triangleq \mathsf{Proc} \\
[\![ff]\!] &\triangleq \emptyset \\
[\![F \wedge G]\!] &\triangleq [\![F]\!] \cap [\![G]\!] \\
[\![F \vee G]\!] &\triangleq [\![F]\!] \cup [\![G]\!] \\
[\![[\alpha]F]\!] &\triangleq [\cdot\alpha\cdot][\![F]\!] \\
[\![\langle\alpha\rangle F]\!] &\triangleq \langle\cdot\alpha\cdot\rangle[\![F]\!] \\
[\![X_{i,j}]\!] &\triangleq \begin{cases} \left(\mathrm{FIX}\, \mathcal{O}_{\boldsymbol{F}_i;\mathbf{f}_i}\right)_j & \text{falls } m_i = \max \\ \left(\mathrm{fix}\, \mathcal{O}_{\boldsymbol{F}_i;\mathbf{f}_i}\right)_j & \text{falls } m_i = \min \end{cases}
\end{aligned}$$

wobei FIX und fix wie in Theorem 6.17 gegeben sind.

Spieltheorie für „Model-Checking“ in 1HML

Wir nutzen eine Form von Spielen, genannt *MC-Spiele*,
um für $p \in \mathsf{Proc}$ und $F \in \mathcal{M}_{\{X\}}$ herauszufinden,
ob $p \models F$ gilt, bzw., ob $p \in [\![F]\!]$.

6.21 Definition (Spielpositionen)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$, mit $\mathcal{M}_{\{X\}}$ definiert über Act.
Dann ist $H^T \triangleq \mathsf{Proc} \times \mathcal{M}_{\{X\}}$ die Menge der *MC-Spielpositionen in* T.
H^T (kurz: H) wird syntaktisch partitioniert in:

- *MC-Angriffspositionen*

$$H_a^T \triangleq \mathsf{Proc} \times \Big(\{ tt \} \cup \big\{ F_1 \wedge F_2 \mid F_1, F_2 \in \mathcal{M}_{\{X\}} \big\} \cup \big\{ [\alpha]F \mid \alpha \in \mathsf{Act}, F \in \mathcal{M}_{\{X\}} \big\} \Big)$$

- *MC-Verteidigungspositionen*

$$H_d^T \triangleq \mathsf{Proc} \times \Big(\{ ff \} \cup \big\{ F_1 \vee F_2 \mid F_1, F_2 \in \mathcal{M}_{\{X\}} \big\} \cup \big\{ \langle\alpha\rangle F \mid \alpha \in \mathsf{Act}, F \in \mathcal{M}_{\{X\}} \big\} \Big)$$

- *MC-Rekursionspositionen*

$$H_r^T \triangleq \mathsf{Proc} \times \{X\}.$$

Oft lassen wir das Superskript T weg, wenn es im Kontext klar ist.

6.22 Definition (Spielzüge)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Die Menge der *MC-Spielzüge in* T ist definiert
als Relation $\rightarrowtail \subseteq H \times H$ durch:

- MC-Angriffszüge $\overset{A}{\rightarrowtail} \subseteq H_a \times H$
  - $(p, \bigwedge_{i \in I} F_i) \overset{A}{\rightarrowtail} (p, F_i)$ für alle $i \in I$
  - $(p, F_1 \wedge F_2) \overset{A}{\rightarrowtail} (p, F_1)$ und $(p, F_1 \wedge F_2) \overset{A}{\rightarrowtail} (p, F_2)$
  - $(p, [\alpha]F) \overset{A}{\rightarrowtail} (p', F)$, falls $(p \xrightarrow{\alpha} p') \in \mathsf{Tran}$.
- MC-Verteidigungszüge $\overset{D}{\rightarrowtail} \subseteq H_d \times H$
  - $(p, \bigvee_{i \in I} F_i) \overset{D}{\rightarrowtail} (p, F_i)$ für alle $i \in I$
  - $(p, F_1 \vee F_2) \overset{D}{\rightarrowtail} (p, F_1)$ und $(p, F_1 \vee F_2) \overset{D}{\rightarrowtail} (p, F_2)$
  - $(p, \langle\alpha\rangle F) \overset{D}{\rightarrowtail} (p', F)$, falls $(p \xrightarrow{\alpha} p') \in \mathsf{Tran}$.
- MC-Rekursionszüge $\overset{R}{\rightarrowtail} \subseteq H_r \times H$
  - $(p, X) \overset{R}{\rightarrowtail} (p, F_X)$, falls $X \overset{m}{=} F_X$ für $m \in \{\max, \min\}$.

Wir definieren $\rightarrowtail \triangleq \overset{A}{\rightarrowtail} \cup \overset{D}{\rightarrowtail} \cup \overset{R}{\rightarrowtail}$.

Für $h \in H$ definieren wir $\mathrm{Reach}(h) \triangleq \{ h' \in H \mid h \rightarrowtail^* h' \}$.

Bemerkung: In Definition 6.22 geben wir explizite Varianten für alternative Syntaxformen an. Wir tun dies im Interesse einer kompakteren Darstellung von Spielgraphen. Hier passt ohnehin die Interpretation der n-stelligen Varianten als Repräsentantensystem nicht mehr, denn Positionen mit logisch äquivalenten Formeln (z.B. $(F_1 \wedge F_2) \wedge F_3$ und $F_1 \wedge (F_2 \wedge F_3)$) haben nicht notwendigerweise die gleichen Spielzüge.

6.23 Definition (Spielstand)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Ein *MC-Spielstand* h in T, ausgehend von einer Startposition $h_{\text{init}} \in H$, ist eine Abbildung

$$h : \mathbb{N} \to H \cup \{\bot\}$$

mit

- $h(0) = h_{\text{init}}$ und
- für alle $i > 0$ mit $h(i) \neq \bot$ gilt $h(i{-}1) \rightarrowtail h(i)$.

Die Anzahl z_h der MC-Spielzüge in h ist gegeben durch:

$$z_h \triangleq \begin{cases} \infty & \text{falls } h : \mathbb{N} \to H \\ \max\{\, i \mid h(i) \neq \bot \,\} & \text{sonst} \end{cases}$$

$\mathcal{H}(h_{\text{init}})$ bezeichnet die Menge *aller* Spielstände ab h_{init}.

6.24 Definition (Spiel)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Ein *MC-Spiel* in T, ausgehend von $h_{\text{init}} \in H$,
ist ein *maximaler* MC-Spielstand h in T, also entweder

- eine unendlich lange Sequenz (mit $z_h = \infty$), oder
- eine endliche Sequenz (mit $z_h \in \mathbb{N}$ Spielzügen), für die es kein $h \in H$ gibt, so dass $h(z_h) \rightarrowtail h$.

Wir schreiben $\mathcal{H}^{\surd}(h_{\text{init}})$ für die Menge aller Spiele ab h_{init}.

6.25 Definition (Spielbewertung)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $X \stackrel{m}{=} F_X$ für $m \in \{\max, \min\}$.
Unendliche MC-Spiele h in T gewinnt

- die Verteidigungsseite, falls $m = \max$;
- die Angriffsseite, falls $m = \min$.

Endliche MC-Spiele h in T gewinnt

- die Verteidigungsseite, falls $h(z_h) \in H_a$.
- die Angriffsseite, falls $h(z_h) \in H_d$.

6.26 Bemerkung MC-Spiele werden von genau einer Seite gewonnen.

6.27 Definition (Spielstrategie)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Eine *Spielstrategie* für MC-Spielpositionen $H_{str} \subseteq H$ ist eine Abbildung $f : H_{str} \to (H \cup \{\bot\})$ mit

- $f(h) \in H$ impliziert $h \rightarrowtail f(h)$, und
- $f(h) = \bot$ impliziert, dass es kein $h' \in H$ gibt mit $h \rightarrowtail h'$.

6.28 Definition (Konsistenz)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $h \in \mathcal{H}(h_{init})$ ein MC-Spielstand in T.
Sei f eine Spielstrategie für H_{str} wie in Definition 6.27.
Dann heißt h *konsistent* mit f,
falls für alle $i \in \mathbb{N}$ mit

- $h(i) \in H_{str}$ und
- $h(i{+}1) \neq \bot$

gilt: $h(i{+}1) = f(h(i))$.

6.29 Definition (Gewinnstrategien)
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$. Sei $h_{init} \in H$.
Die Angriffsseite besitzt eine *universelle Gewinnstrategie* f für H_{str} ab h_{init}, wenn $(\mathrm{Reach}(h_{init}) \cap H_a) \subseteq H_{str} \subseteq H_a$ und wenn sie alle MC-Spiele in $\mathcal{H}^{\surd}(h_{init})$, die konsistent mit dieser Strategie sind, gewinnt.
Die Verteidigungsseite besitzt eine *universelle Gewinnstrategie* f für H_{str} ab h_{init}, wenn $\big(\mathrm{Reach}(h_{init}) \cap H_d\big) \subseteq H_{str} \subseteq H_d$ und wenn sie alle MC-Spiele in $\mathcal{H}^{\surd}(h_{init})$, die konsistent mit dieser Strategie sind, gewinnt.

6.30 Bemerkung
Eine universelle Gewinnstrategie für die Verteidigung nennen wir auch *universelle Verteidigungsstrategie*. Eine universelle Gewinnstrategie für den Angriff nennen wir auch *universelle Angriffsstrategie*.

6.31 Theorem (Theorem 6.3 in [AILS07])
Sei $T = (\mathsf{Proc}, \mathsf{Act}, \mathsf{Tran})$.
Sei $X \overset{m}{=} F_X$ für $m \in \{\max, \min\}$.
Sei $p \in \mathsf{Proc}$ und $F \in \mathcal{M}_{\{X\}}$.
Dann gilt:

- $p \models F$ genau dann, wenn es
  eine universelle Verteidigungsstrategie für MC-Spiele ab (p, F) gibt.
- $p \not\models F$ genau dann, wenn es
  eine universelle Angriffsstrategie für MC-Spiele ab (p, F) gibt.